Milestone-A API Access and Test Preparation

Melbourne Urban Forest public tree data

1 Document details

Field	Value
Prepared by	Darvin Anorma Mangampo
Assessment period	Planning began 12 October 2026. Preflight and report prepared 13 October 2026 (UTC+8)
Repository	melbourne-urban-forest-api-tests
Target	City of Melbourne Urban Forest tree data and dataset metadata
Readiness assessment	API access was confirmed. The test plan and eight-row matrix set out the test conditions.
Document date	13 October 2026

2 Purpose and project context

On 12 October 2026, I began narrowing the project to Melbourne's Urban Forest data after completing The Test Tribe Playwright course. The service is public, live and read-only. It supports a practical consumer workflow. A user can list a few trees, move to the next page, filter by genus and look up one of those trees again. I first checked whether Playwright could access the API from my project environment and return fields suitable for verification.

I based the eight checks on the API documentation, the preflight results and how someone would use the tree data. The log shows the statuses and fields I examined. It does not show that the eight-condition suite has passed. I started this 2026 API testing project as part of my upskilling to apply what I learned in the Playwright course, separate from the work I did in my previous role.

3 Results of the 13 October preflight

On 13 October 2026, I used Playwright 1.64.0 with Node.js 24.19.0 on my computer to send twelve GET requests, pausing briefly between them. All twelve responses were JSON. Eight returned HTTP 200. Four returned HTTP 400 for deliberately invalid or out-of-range inputs. No request returned HTTP 403, HTTP 429 or an access challenge.

Area examined	Observed result
Tree data and metadata	The tree data endpoint returned fields including com_id and genus. Both endpoints reported 82,064 trees in this run.
Tree lookup	I looked up public com_id 1013381 again. Its com_id and genus (Ulmus) matched the source row. An unused ID returned 200 with an empty result.
Filtering and pagination	The filter and sorted-page requests returned HTTP 200. The log shows a true Ulmus comparison flag and no repeated IDs. It does not retain the filtered or second-page row counts, so the full suite must still confirm that both responses contain rows.
Input handling	limit=100 returned HTTP 200 with 100 rows. limit=101 returned HTTP 400 with JSON. Unknown fields in where and order_by each returned HTTP 400 with a field-specific JSON message.

These figures come from one run. The tree total and the tree used for the lookup can change as the public dataset changes.

4 API access and test preparation

In the preflight, I received JSON responses and checked the fields used in the matrix. The lookup matched the tree ID and genus across two responses, and the tree totals agreed during the run. The script labelled the outcome ready. The saved filter and page comparison flags do not establish that both results contained rows because the log did not retain their counts.

The test plan and eight-row matrix describe the API checks. They cover response shape, two sorted pages, lookup by com_id, genus filtering, the 100/101 page-size pair, invalid query fields, agreement between the data and metadata totals, and an unused tree ID. Each row links a request to an expected result and explains whether that expectation comes from the provider reference, the dated preflight or a comparison between responses.

5 Pagination limit observation

The request at offset + limit = 10,000 returned HTTP 200 with 100 rows. At 10,001, the API returned HTTP 400 and said the sum must be at most 10,000. The published Explore API reference says the sum should be less than 10,000. I have kept this as a dated documentation observation outside the eight conditions in the matrix. T05 covers the separate page-size pair of limit=100 and limit=101. One run does not establish a provider defect.

6 Boundaries of this evidence

No full test suite, browser check, data-changing API request, load test or security test was run in Milestone-A. The equal tree total is a snapshot. It does not mean two live responses will always agree. The saved log shows that the genus filter and second sorted-page requests returned HTTP 200, but it does not include the number of rows in either response. Their comparison flags can be true even when a response contains no rows. The saved log shows HTTP 200 for those requests, but without row counts I cannot tell whether the filtered and second-page responses contained trees. An unexpected response alone does not establish a problem with the API. The request, run date and API documentation need to be checked first. This report uses public API observations only. It contains no client data, credentials or private employment files.

No client requirements were supplied for this project. I used the published API reference for documented expectations and compared fields and totals across responses from the same run. These checks do not verify whether the source tree inventory is accurate. A result seen only in the preflight remains an observation to review again.

7 Supporting files and public references

The files below support this Milestone-A report. The preflight script contains the API requests I ran, and the JSON log contains their dated results. The test plan explains the scope and approach. The matrix lists the checks and the basis for each expected result.

Repository path	Purpose
scripts/preflight.cjs	The Playwright requests I ran during the dated check.
evidence/Milestone-A/preflight-results.json	Twelve request paths, statuses, response types and key values.
evidence/test-plan.md	The scope, approach, data, entry and exit criteria, and risks.
evidence/test-matrix.md	Eight test conditions and the basis for each expected result.
evidence/Milestone-A/README.md	Milestone-A navigation and rerun instructions.

The public sources are the City of Melbourne Urban Forest dataset and the Huwise Explore API v2.1 reference.

8 Conclusion

On 12 October 2026, I chose Melbourne’s Urban Forest data for this project. On 13 October, I used Playwright to send twelve read-only requests to its live API. The responses included usable tree fields and clear results at the page-size limit, so I retained the API as the project subject.

Milestone-A confirms API access and explains the scope and approach for the checks. It does not report a full-suite result. The missing row counts limit what can be said about the filtered and second-page responses. The difference between the published paging limit and the response at exactly 10,000 remains a dated observation rather than a confirmed provider defect.